Sets Are Rules, Not Buckets: Intensional Programming in C++23 via `dedekind`*

The `dedekind` Authors

April 24, 2026

Abstract

We present `dedekind` [2], an open-source C++23 library that embeds formal set-theoretic and categorical structures directly into the type system, available under the Apache-2.0 licence. The central design rule is *intensional first, realize it when you mean it*: a set is represented as a predicate $P : A \to \Omega$ over an ambient species A, not as a heap-allocated container of values. This comprehension view, rooted in Lawvere's Elementary Theory of the Category of Sets (ETCS) [3], has two practical consequences that this paper explores:

- Set operations (intersection, complement, union) compose as predicate combinators at compile time, enabling the LLVM backend to perform *structural pruning*—collapsing logically empty intersections into zero-instruction machine code.
- Numeric realization is deferred to explicit boundaries, cleanly separating abstract algebraic reasoning (e.g. over the naturals $\mathbb{N}$) from hardware-constrained IEEE 754 floating-point [4] semantics.

We motivate the design from the library-implementer's perspective: mathematically faithful libraries in compiled languages have traditionally paid for algebraic invariants either in silent bugs or in runtime checks, and we argue that recent C++23 advances (concepts, non-type template parameters, named modules) let those invariants live at the type level instead. We identify *Axiomatic Systems Programming* as a nascent paradigm in which the compiler—explicitly *not* a theorem prover, but a structural gate—rules out whole classes of algebraic-law violations at translation time and enables whole classes of structural optimizations that exploit the ruled-out failure modes. As the centrepiece existential proof, we show that a textbook linear-programming instance reduces at compile time to its optimal vertex as a typed constant—the six candidate solves, six feasibility checks, and six objective comparisons all vanish in the emitted IR, which contains only the literal return of the optimum's coordinates. Running the same reduction over the dual-number ring $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$ additionally recovers exact sensitivity analysis with no separate derivative pass, recognising forward-mode automatic differentiation as a sibling compile-time reduction in the sense of Elliott [5]. Both reductions share a single mechanism: the feasible polytope is a set-as-predicate, finite vertex candidates enumerate as pairs of binding constraints, and objective-directed argmax collapses the candidate set to a singleton. The same discipline extends to polynomial calculus: the Fundamental Theorem of Calculus (Part II) is discharged at translation time, with the compiler running the trapezoidal quadrature against a closed-form antiderivative and concluding inside a `static_assert`.

The thesis this paper argues, in a single sentence: *named algebraic structure at the type level, discharged by the compiler, narrows the gap between abstract mathematics and machine code without a separate proof assistant.*

*Draft intended for submission to *Programming* [1], the journal for programming-related research. The paper's scope is deliberately narrower than the companion project report: this document focuses on the sets-as-predicates thesis and compile-time structural pruning; the broader library (categorical foundations, algebra tower, numeric tower, geometry, linear algebra, morphologies, Python facade) is documented in the companion Report distributed alongside the source tree.

Keywords: C++23; concepts; non-type template parameters; compile-time verification; Axiomatic Systems Programming; type-directed collapse; linear programming; forward-mode automatic differentiation; edge computing; extreme edge computing (XEC); green edge computing; embedded machine learning; sustainable computing.

1 Introduction

1.1 Motivation: Mathematical Faithfulness in a Compiled Library

A library implementer who wants to expose mathematical abstractions in a compiled language faces a recurring tension. Algebraic structures—groups, rings, fields, vector spaces—are defined by laws: associativity, commutativity, identities, inverses. But the concrete carriers a compiled language offers violate those laws conditionally: a 32-bit signed `int` is not an additive group (overflow, `INT_MIN` has no negation), and IEEE 754 `double` [4] is not associative under addition. The library author's options are to ignore the divergence (silent bugs on specific inputs), police it at runtime (hot paths pay), or encode it in the type system so the compiler can police and exploit it. This paper is about what happens when a library takes the third option seriously, in standard C++23.

The third option requires a type system strong enough to name algebraic structure and to discriminate between carriers that satisfy a law and those that do not. Haskell type classes [6] and dependently-typed languages such as Coq and Agda [7] demonstrate the expressiveness; both impose runtime or toolchain costs that are hard to justify when the library's consumers are writing systems code. C++23 now offers a pragmatic middle ground: *concepts* (the compile-time analogue of Haskell type classes) check structural membership at compile time without nominal declarations, *non-type template parameters* (NTTPs—template arguments that are values, not types) let lambdas and other compile-time data appear in type signatures, and *named modules* enforce a strict directed-acyclic build graph. The compiler becomes both a *verification gate*—violating an algebraic law is a type error, not a runtime exception—and an *optimization engine*—the LLVM backend exploits the structural knowledge inherited from the discharged proof obligations.

Our posture is explicit: `dedekind` is a systems-programming library that treats mathematical concepts as first-class compile-time objects. The library is implemented in C++23, and the compile-time concept machinery is its primary interface; the consumer we design for is another C++ author who would otherwise bolt ad-hoc structure onto unverified numeric types. Choosing to land the semantics at the compile-time layer—rather than as runtime abstractions, or as a DSL lowered to a runtime solver—is a design commitment that shapes what the compiler can do for the user and what it cannot.

Thesis. In one sentence: *named algebraic structure at the type level, discharged by the compiler, narrows the gap between abstract mathematics and machine code without a separate proof assistant.* Every section of the paper serves that sentence. Sets-as-rules, intensional-first, computability-tier collapse, the LP and AD and FTC reductions—these are the *means* by which the thesis is discharged, and the mechanical witnesses (`static_asserts` and IR fixtures) are the evidence that it is discharged in practice.

What is new. Compile-time verification of algebraic structure is not new in the abstract. Agda and Coq host it natively; Liquid Haskell attaches refinement predicates to types; C++ template-metaprogramming traditions (most visibly Boost.Hana [8] and CTRE [9]) have demonstrated type-level computation at scale for over a decade; autodiff libraries such as Ceres-Solver [10] and ENOKI [11] have carried fragments of the dual-number story into production C++. What *is* new, we claim, is that the full technique—named algebraic structure, discharged by the compiler, with the LLVM backend carrying the discharge through to optimised machine

code—is now available, today, in standard C++23, without a proof assistant, a custom type checker, or a bespoke language extension. `dedekind` is the existence proof. The viewpoint on which it rests we name *Axiomatic Systems Programming*: a nascent engineering discipline of practising, not merely describing, type-level structural verification in mainstream compiled code. The discipline can be lifted, with modest adjustment, to any language whose type system can name a predicate over compile-time data and check it mechanically (concepts-style Java/C#, strictly-typed Python, the relevant parts of Rust). The contribution of this paper is the first rigorous articulation of that discipline, together with a worked library that discharges its claims mechanically.

Intended target: CPU-first and embedded machine learning. One concrete application domain motivates the discipline and runs through the showcases of Section 5: computation that leaves the data centre and moves to the edge. In CPU-first and embedded machine-learning deployments—vehicles, robotics, scientific instruments, industrial controllers, the wider edge computing [12] landscape—the operating point is precisely the one our Compiler Wall (Section 5.9) is tuned for: small fixed-topology problems, hard constraints on runtime footprint, a strong preference for deterministic arithmetic, and a toolchain no heavier than the system's `clang++` already is. Model Predictive Control, Kalman-style sensor fusion, quaternion-based pose math, small-model inference with gradients—these are fixed-shape computations whose coefficients and dimensions are known at compile time; existing embedded ML runtimes (TFLite Micro [13], ONNX Runtime, microTVM) schedule them on top of a heavyweight interpreter or a code-generator with its own dependency closure. `dedekind` sits one layer down: it is the *compile-time substrate* against which such primitives can be written directly in standard C++23, with the structural discipline this paper articulates, and with no runtime at all. The Kalman filter of Section 5.6 is the paper's worked embedded primitive; MPC and micro-inference are the natural successors.

The compiler as a poor man's theorem prover. A pedagogical framing that runs through this paper: we are using `clang++` as a poor man's theorem prover. The compiler is not searching for proofs—it has no tactic language, no resolution procedure, no proof assistant in the loop. What it *does* have is a type system strong enough to name a predicate, a `static_assert` strong enough to discharge it, and an LLVM backend that will happily remove what has already been decided. The library author supplies narrow, mechanically checkable proof obligations; the compiler discharges them; the optimiser then deletes the code that would have re-checked them at runtime. As a welcome side effect, the same machinery produces a family of composable library combinators (set meets, cuts, halfspaces, LP reductions) whose laws are verified at translation time and whose trivial cases collapse to zero instructions.

Most of what follows is therefore *not* specific to C++. Concepts have close analogues in Java (sealed types plus generic constraints), C# (constrained generics, augmented by Roslyn source generators and analysers), and Python (PEP 484 type hints under a strict `mypy`/`pyright` pass). Any language whose type system can name a predicate over compile-time data and check it mechanically is a candidate host for the same technique. What C++23 contributes on top is the LLVM backend sitting downstream of the type checker: discharged proofs do not only *verify*, they also *optimise*, and the connection between the two is what lets the LP vertex collapse to a literal return rather than to a skipped runtime check.

What this buys, concretely. From one underlying idea ("name the structure in the type, let the compiler discharge the laws") and a small handful of primitives (concepts, NTTP-carried predicates, halfspaces, cuts), the library builds up, at compile time: a numeric tower ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ via Dedekind cuts, $\mathbb{C}$, dual numbers); a small linear-algebra layer (`Vec2`, `Invertible2x2`

with closed-form Cramer inverse); a calculus layer (numerical, polynomial-symbolic, and dual-number forward AD, each a realisation of Elliott's (f, f') product); and an optimisation layer (the LP vertex enumerator of Section 5.2). No runtime solver ships with any of it. The breadth is part of the contribution: the technique scales from a single ring concept all the way up to an LP whose optimum is a `static_assert`-checked literal in the emitted object file.

A worked example in advance. Section 5.2 presents the paper's centrepiece: a textbook linear-programming instance whose objective vector and constraint polytope are named at the type level reduces at compile time to its optimal vertex as a typed constant. The six candidate solves, six feasibility checks, and six objective comparisons vanish in the emitted IR, which contains only the literal return of the optimum's coordinates. No LP solver runs; the compiler has already performed the active-set enumeration. Running the same reduction over the dual-number ring (Section 5.3) additionally recovers exact sensitivity analysis, recognising forward-mode automatic differentiation as a sibling compile-time reduction in the sense of Elliott [5]. Both reductions are witnessed by `static_assert` (C++'s compile-time assertion—if the predicate is false, the program fails to build) at instantiation and by checked-in LLVM IR fixtures that fail CI on optimiser drift. A third, smaller showcase (Section 5.5) lifts the same discipline to polynomial calculus: the formal derivative of a rational polynomial is a `static_assert`-checked identity, and the Fundamental Theorem of Calculus (Part II) is discharged at translation time against a closed-form antiderivative.

The three claims this paper substantiates.

1. Sets are *rules* (predicates), not *buckets* (containers).
2. Computation is *intensional until explicitly realized*.
3. The compiler is used as a *verification and optimization engine* for algebraic invariants.

The remainder of the paper develops these three claims and demonstrates each by a reduction that compiles to a constant.

1.2 How to Read This Paper

Code and mathematics appear side by side throughout. That pairing is the whole point. Most of the interesting objects in this paper—Dedekind cuts, rings, dual numbers, polytopes, active sets, ring homomorphisms— have two faces: a math notation most readers have met at some point, and a C++ type most readers will find easier to trace. Put both on the page and you get a kind of binocular vision: whichever face is clearer for you carries the meaning, and the other face falls into place.

No prior familiarity with C++23 templates or with the specific mathematical objects is required. When something non-obvious appears—a non-type template parameter, a ring homomorphism, a co-Kleisli arrow—it is unpacked on first use, often by showing the corresponding C++ idiom next to the math definition and inviting the reader to read them as two names for the same thing. If you are at home with Haskell's type classes, Agda's dependent types, or Coq's proof terms, many of the moves will feel familiar; if not, the C++ side is the on-ramp.

1.3 Paper Structure

Section 2 motivates the choice of C++23 as the implementation substrate and introduces the *Axiomatic Systems Programming* paradigm. Section 3 explains the intensional-first design rule and the tension between abstract algebraic numerics and IEEE 754 hardware floating point. Section 4 develops the view of sets as predicates over ambient species and shows how set operations compose symbolically. Section 5 presents compile-time structural reductions: halfspace

pruning, linear programming as a reduction to a typed vertex, and forward-mode automatic differentiation as a sibling reduction over the dual-number ring. Section 6 surveys related work. Section 7 concludes.

2 Axiomatic Systems Programming

2.1 The Substrate: C++23

The *two-language problem* [14] is the observation that research code is written in an expressive high-level language (Python, R, Julia) while performance-critical execution is delegated to a separately maintained C or C++ backend. Bridging tools such as `nanobind` [15] close the *binary* gap but not the *semantic* gap: the C++ backend cannot reason about high-level mathematical invariants defined in the Python layer.

Recent advances in C++ change this calculus. Three C++23 features are central:

Concepts allow *structural typing*: a type satisfies a concept if it provides the required operations and relations, regardless of its name. This is the compile-time analogue of Haskell type classes, but with zero-cost dispatch and no runtime dictionary.

Non-type template parameters (NTTP) allow lambdas—including predicate functions—to appear as template arguments. A set-builder predicate can therefore be part of the *type*, not merely a runtime value passed at construction.

Named modules enforce a strict directed-acyclic build graph, preventing circular dependencies between algebraic layers and caching verified structural invariants as Binary Module Interface (BMI) files.

Together, these features let the compiler rule out whole classes of algebraic-law violations at translation time and exploit the resulting structural knowledge for optimization. This is *distinct* from theorem-proving: the compiler discharges the proof obligations it is given, it does not search for new ones.

2.2 Structural vs. Nominal Typing: The Juliet Posture

Three distinct schools of structural identification appear in the history of programming languages. Understanding their differences clarifies why C++23 concepts represent a genuine advance.

Nominal typing. Traditional object-oriented frameworks require a type to *declare* its membership. A class is a `Ring` because it inherits from a base class named `Ring`, or implements an interface so named. The *name* is the proof of membership. The compiler checks lineage, not behaviour. A type that provides every ring operation but neglects to inherit the right base class is silently excluded; a type that inherits the right class but provides broken operations is silently accepted.

Duck typing. Dynamic languages (Python, Ruby, early JavaScript) use a runtime structural check colloquially known as *duck typing*: "if it walks like a duck and quacks like a duck, it is a duck." Membership is determined at the call site by testing whether the required method exists and can be invoked. This is genuinely structural—it cares about behaviour, not name—but the check happens *at runtime*, too late to drive compile-time optimization, and errors surface only when a missing method is actually called. A set represented as a Python `list` and tested by `in` is duck-typed in this sense: the membership check succeeds or fails at the point of evaluation, with no opportunity for the compiler to discover that the set is empty or that two conjoined predicates are contradictory.

Static structural typing—the Juliet Posture. C++23 concepts combine the structural insight of duck typing with the static, compile-time guarantees of nominal typing—without requiring a declaration of intent. We call this the *Juliet Posture*, after the observation that a rose by any other name would smell as sweet [16]: the library cares about the *scent* (structural invariants), not the *lineage* (declared name).

```
// Nominal (pseudocode): recognised by declared lineage.
struct Ring { /* virtual operations, must opt in explicitly */ };
struct MyInt : Ring { /* ... */ };

// Duck typing (pseudocode): recognised at runtime by the call site.
//   def generic_sum(a, b):       # Python: fails only when called
//       return a + b             # no compile-time guarantee

// Juliet Posture (C++23, the actual dedekind concept):
template <typename T>
concept IsRingLike = requires(T a, T b) {
  { a + b  } -> std::same_as<T>;
  { a - b  } -> std::same_as<T>;
  { -a     } -> std::same_as<T>;
  { a * b  } -> std::same_as<T>;
};

// int satisfies IsRingLike without any base class, checked at compile
// time: the operators exist. But the operational concept is deliberately
// weaker than the algebraic laws --- see lst:species --- so a separate
// trait lookup filters out carriers whose + is not actually associative.
static_assert(IsRingLike<int>);           // operators exist
static_assert(IsRingLike<Rational<long>>);
static_assert(!is_associative_v<int, std::plus<int>>);  // but + is not a monoid
```

Listing 1: The Juliet Posture: structural recognition checked statically, without requiring declared lineage. The operational concept from `dedekind.algebra:ring`.

The Juliet Posture delivers the best properties of both predecessors: structural sensitivity (behaviour, not name) and static verification (compile time, not runtime). Crucially, because the concept check is a compile-time proof obligation, the compiler can *act on it*: it can specialize code paths, infer additional algebraic properties from the trait registry, and ultimately prune logically empty set expressions to zero machine instructions—none of which is possible with runtime duck typing.

2.3 The Compiler as Structural Gate and Optimization Engine

Two durable uses of a compile-time type system underlie this paradigm, neither of which requires the compiler to be a theorem prover:

1. **Ruling out entire classes of errors.** Encoding algebraic laws as concept requirements makes every instantiation of a generic template an implicit proof obligation discharged at translation time. Failed obligations are type errors, not runtime exceptions. The obligation itself is finite, decidable, and mechanical—the compiler *checks* it, but does not *search for* it.

2. **Enabling entire classes of optimizations.** The LLVM backend exploits the structural knowledge inherited from discharged obligations. When a set is defined by a predicate and two predicates are logically contradictory, the compiler determines at translation time that the resulting intersection is empty and emits no machine instructions for it. This is what we term *structural pruning*, and it is the subject of Section 5.

Both uses are bounded. clang's type checker is not a theorem prover: it has no dependent types, no universal-quantification inference, no proof-search engine. What it does have is `static_assert`, `requires`-clauses, and `constexpr` evaluation—enough to verify a predetermined proof sketch, not enough to generate one. The paradigm claims the reliable half: ruling out, and optimizing under, the classes of structural failures the programmer has named in the type system. The Curry–Howard correspondence between propositions and types is suggestive here, but its full force (dependent products, inductive proofs, universe hierarchies) is not available in C++23; what *is* available is enough to anchor the two uses above.

Bounded yet open-ended

Each use is bounded *individually*, but the paradigm as a whole exhibits a productive open-endedness worth naming. Every class of errors ruled out, and every class of optimizations enabled, tends to expose a next class that the current type machinery does not handle but that the library's structural vocabulary names clearly enough to become a tracked, narrowly-scoped follow-on. In our setting the four axes listed in Section 5.8 are each discovered rather than engineered: they fall out of the halfspace reductions of Section 5 almost by construction.

Penrose offers a related observation about Gödel's incompleteness theorem in *The Road to Reality* [17]: a formal system that closes at one level thereby motivates a next level, not as an exhaustion but as a natural successor. Our setting is more benign than Penrose's philosophical one—the issue tracker plays the role he reserves for the mind's eye, and the successor steps are ordinary software work rather than exits from formal systems—but the pattern is the same. The closest formal analogue in the mathematical-logic literature is Turing's ordinal logics [18], where each stage's consistency feeds the next stage's formation rule. We do not need the full apparatus: the point for a systems-programming paradigm is that closure-at-stage-n is a generative signal for stage-$n{+}1$, rather than a terminal achievement.

3 Intensional First, Realize When You Mean It

3.1 The Design Rule

The central methodology of `dedekind` is captured by a single rule:

> *Intensional first, realize it when you mean it.*

The thrust of this rule is to leverage *lazy evaluation*—both at the type level (compile-time symbolic substitution) and at the value level (deferred numeric approximation)—as a principled defence against *Premature Materialization*: the loss of mathematical structure that occurs when a computation is forced to a concrete representation earlier than necessary.

An *intensional* description specifies a mathematical object by its defining properties. An *extensional* representation enumerates its members or computes its value. A halfspace $\{x \in \mathbb{Q} \mid x \leq 4\}$ is intensional; the list of its rational points between 0 and 4 is extensional; and a function that materialises the former into the latter is the auditable crossing of that boundary. The Dedekind cuts of Section 3.3—where a real number is named by the pair of rational sets below and above it—push the same distinction to its limit: the real is intensional forever, and only bounded-precision queries cross the boundary. This framing is close in spirit to intensional database semantics (for example, Majkić's intensional FOL treatment for peer-to-peer data integration [19]), but our engineering contrast with mainstream SQL is explicit: SQL evaluation is typically defined over already-realized relations, while `dedekind` preserves predicate-level structure in the type system and defers materialization to an explicit realization boundary.

3.2 The Tension: Abstract Naturals vs. IEEE Numerics

Here is the tension in one sentence: the `int` in your source file is not the $\mathbb{Z}$ in your textbook, and everybody quietly agrees to pretend it is. The ring $\mathbb{Z}$ obeys a few laws:

- Associativity of addition: $(a + b) + c = a + (b + c)$.
- Existence of additive inverse: $\forall a, \exists -a$ such that $a + (-a) = 0$.
- No upper or lower bound.

A 32-bit signed `int` obeys *none* of them exactly: addition overflows, negation of `INT_MIN` is undefined behaviour, and the representable range is finite. IEEE 754 `double` [4] is further out still: addition is non-associative because rounding happens every step, and `NaN` and $\pm\infty$ puncture the law of excluded middle in the exact way Section 4 will ask the subobject classifier Ω to repair. A generic algorithm that silently assumes the ring laws on these carriers is, in the literal technical sense, wrong—it just happens not to be wrong on most inputs.

`dedekind` encodes these divergences explicitly in the `:species` trait registry:

```
// Variable template: is_associative_v<T, Op> is the structural trait.
// The library partial-specialises it per-carrier, per-operation.

// Unsigned integrals are associative under +  (modular arithmetic).
template <std::unsigned_integral T>
inline constexpr bool is_associative_v<T, std::plus<T>> = true;

// Signed integrals are REJECTED: overflow of a signed integer is
// undefined behaviour, so the addition operator is not a law-abiding
// monoid on signed carriers.
template <std::signed_integral T>
inline constexpr bool is_associative_v<T, std::plus<T>> = false;

// IEEE double is rejected for a different reason: rounding breaks
// bit-exact associativity.
template <>
inline constexpr bool is_associative_v<double, std::plus<double>> = false;

static_assert(is_associative_v<unsigned int, std::plus<unsigned int>>);
static_assert(!is_associative_v<int,         std::plus<int>>);
static_assert(!is_associative_v<double,      std::plus<double>>);
```

Listing 2: The species registry records algebraic properties—and their absence. Signed `int` is explicitly rejected as associative under `+` because signed overflow is undefined behaviour.

Generic algorithms that require associativity will fail to compile for `int` or `double` alike, unless an explicit numeric policy (modular semantics on signed overflow, compensated summation, an interval-arithmetic wrapper, or a lift to `Rational<long>`) is supplied. The realization boundary is the point at which the programmer chooses which policy to apply.

3.3 Dedekind Cuts as the Canonical Example

Here is Dedekind's move, the library's namesake. To name a real number, don't try to write it down. Instead, partition the rationals into those strictly below it and those at or above it. That partition *is* the number. You don't need to compute the real; you just need to be able to tell, for any rational, which side it lands on. Formally, $r \in \mathbb{R}$ is the pair (L, U) with $L = \{q \in \mathbb{Q} \mid q < r\}$ and $U = \{q \in \mathbb{Q} \mid q \geq r\}$.

The construction translates directly: the two sets are type-level predicates, and *the predicate is the number*. The library's canonical worked example is Dedekind's own motivating irrational,

$\sqrt{2}$: a rational q is in the lower cut iff $q^2 < 2$. That is a decidable predicate over $\mathbb{Q}$, so the cut is a finite compile-time object—no approximation, no series summation, just a sentence in the ambient rational logic:

```
template <typename Q>
constexpr auto Sqrt2_Symbolic() {
  const dedekind::sets::Ω<Q, TernaryLogic> universe{};
  // The lower cut L_sqrt2 = { q in Q | q^2 < 2 }.
  return ambient_set<Q>([universe](const Q& q) {
    if constexpr (std::floating_point<Q>) {
      if (std::isnan(q)) { return Ternary::Unknown; }
    }
    const auto in_cut =
        (q * q < static_cast<Q>(2)) ? Ternary::True : Ternary::False;
    return TernaryLogic::AND(universe(q), in_cut);
  });
}
```

Listing 3: $\sqrt{2}$ as an intensional lower cut over $\mathbb{Q}$, from `dedekind.numbers:symbolic`. The predicate is the number.

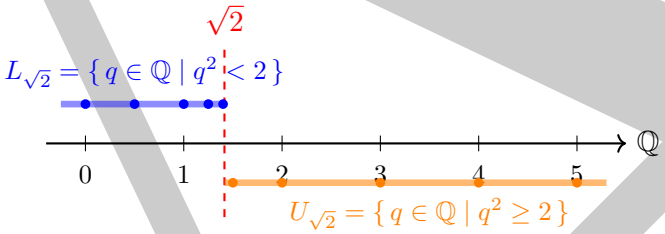

Figure 1: A Dedekind cut names $\sqrt{2}$ by partitioning $\mathbb{Q}$. The lower half $L_{\sqrt{2}}$ (blue) and upper half $U_{\sqrt{2}}$ (orange) are both compile-time predicates. In `dedekind`, the real number *is* the partition: no approximation is stored, no square root is computed, and the compiler still knows everything about $\sqrt{2}$ that the predicate $q^2 < 2$ expresses.

Notice what *hasn't* happened. No floating-point approximation of $\sqrt{2}$ has been computed. No Newton iteration has been run. Asking "is $\frac{7}{5}$ less than $\sqrt{2}$?" reduces to evaluating $\frac{49}{25} < 2$, which is true, at translation time—the answer is a structural check on the predicate rather than a numerical comparison. Approximation to a concrete floating-point value is available later, when the programmer asks for it at a specific precision. Not before.

This is the intensional-first rule applied to the slipperiest objects in elementary mathematics. A real number named as a predicate can have its realisation deferred indefinitely, without losing the ability to reason about it.

3.4 Carriers: From Exact Rationals to Packed Integers

The intensional-first rule applied to *every* numeric role at once is spelled out in three tables. Table 1 catalogs the *exact* side: the library's own intensional carriers for each mathematical role, and the algebraic concept each realises. Table 2 catalogs the *primitive* side: the `std`-library types the hardware runs natively, and the concept each of them carries (with the usual IEEE / overflow caveats). Table 3 is the state-machine that connects the two: given a role, which named embedding or realisation arrow moves you from exact to primitive, and which moves you back. Rows or cells marked with † point at concepts introduced under the concept-cleanup follow-up work; rows or cells marked with ‡ point at carriers whose $_0$-escalating variant is planned but not yet shipped.

Table 1: **Exact carriers.** The library's intensional types, one per mathematical role, together with the algebraic concept each satisfies. These are the carriers a disciplined engineer writes the *reference implementation* against — laws are provably satisfied, not approximated.

Role	Exact carrier	Concept satisfied
$\mathbb{B}$	`bool` under $\wedge, \vee$	`IsBoolean`
$\mathbb{F}_2$	`bool` under $(\oplus, \wedge)$	`IsGaloisField<bool,^,&>` (order 2)
$\mathbb{F}_{64}$	`F64` ($\mathbb{F}_2[x]/(x^6{+}x{+}1)$, 6-bit in `uint8_t`)	`IsGaloisField<T,+,*>` (order 64)
$\mathbb{F}_2^{64}$	`uint64_t` under $\oplus$, scalar action `Bit2U64Action`	`IsVectorSpace<uint64_t, bool>`
$\mathbb{N}$ bounded	`ExtensionalCardinal<N>`	`IsCyclicGroup`†, `IsAbelianGroup<T,+>`
$\mathbb{N}$ unbounded	`Cardinality`‡ (variant over $_0$)	`Monoid_`$\mathbb{N}$
$\mathbb{Z}$ bounded	`SignedExtensionalCardinal<N>`	`IsAbelianGroup<T,+>`, `IsRing`
$\mathbb{Z}$ unbounded	`SignedCardinality`‡	`Group_`$\mathbb{Z}$
$\mathbb{Q}$	`Rational<SignedExtensionalCardinal<>>`	`Field_`$\mathbb{Q}$
$\mathbb{R}$ symbolic	`LowerCut<Rational>` (Dedekind cut)	`Continuum_`$\mathbb{R}$ (predicate-only)
$\mathbb{C}$	`Complex<Rational<SignedEC<>>>`	`Algebra_`$\mathbb{C}$
$\mathbb{D}$ (dual)	`Dual<Rational<SignedEC<>>>`	`IsRing<T,+,*>`, $\varepsilon^2 = 0$

Table 2: **Primitive carriers.** The `std`-library types the hardware runs natively, with the algebraic concept each carries under its C++-standard arithmetic semantics. These are the carriers a disciplined engineer drops to for performance, *after* verifying the reference implementation on the exact side.

Primitive type	Arithmetic semantics	Concept satisfied
`bool`	Boolean algebra under $\wedge, \vee, \neg$	`IsBoolean`
`unsigned long`, `size_t`	Modular, wraps at 2^N	`IsCyclicGroup`†, `IsAbelianGroup<T,+>`
`int`, `long`	Partial (UB on signed overflow)	`IsPartialAbelianGroup`†
`int8_t`, `int16_t`	Partial + SIMD-vectorisable	`IsPartialCyclicGroup`†
`float`, `double`	IEEE 754, rounded, non-associative	`Continuum_`$\mathbb{R}$ (non-assoc. `+`)
`std::complex<double>`	Cartesian pair over IEEE `double`	`Algebra_`$\mathbb{C}$ (non-assoc.)

Table 3: **Crossings state-machine.** The named morphisms that move between exact and primitive sides, per role. Each row shows *realise* (exact $\to$ primitive: drop exactness for speed) and *embed* (primitive $\to$ exact: lift a machine value into the certified carrier). A dash marks a direction that requires going through an intermediate role.

Role	Realise: exact $\to$ primitive	Embed: primitive $\to$ exact
$\mathbb{B}$	(identity)	(identity)
$\mathbb{N}$	`.realize_to_size_t(sentinel)`	`embed_unsigned_integral<N>(v)`
$\mathbb{Z}$	`static_cast<int64_t>(x)`	`embed_signed_integral<Z>(v)`
$\mathbb{Q}$	`.resolve()` $\to$ `double`	`Rational<Z>(num, den)`
$\mathbb{R}$ symbolic	evaluate cut at a rational bound	(intensional; no simple lift)
$\mathbb{R}$ policy	unwrap `IEEE<double>`	wrap in `IEEE<double>`
$\mathbb{C}$	per-component `.resolve()`	`Complex<R>(re, im)`
$\mathbb{D}$ (dual)	`.val` (primal only; tangent discarded)	`Dual<F>(v, d)`

The three tables together are a migration map. The disciplined path is: stay on Table 1 for the reference implementation, use Table 3 to cross over where performance demands, and land in Table 2 with an honest, named set of caveats. The workflow looks like this:

1. **Start correct.** Write the algorithm against the concept column using the exact intensional carrier. For arithmetic- heavy code this usually means `Rational<SignedExtensionalCardinal<>>`: an arbitrary-precision exact rational whose laws are bit-for-bit provable. The code is slow (every operation is an N-limb rational reduction), but it is a reference implementation you can trust.

2. **Verify.** Run the existing test suite. Exact rationals let every `static_assert` be a provable identity rather than a floating-point tolerance check. Regressions surface as compile errors, not as silent drift.

3. **Profile.** Identify the inner loops where arithmetic cost dominates.

4. **Cross over.** In those loops only, substitute the native carrier on the right-hand column. The generic code compiled against `Field_`$\mathbb{Q}$ will also bind to `double` through `Continuum_`$\mathbb{R}$, modulo the policy gates noted in the concept cell (e.g. loss of associativity under `+`, which some algorithms tolerate and others do not). The cost of this step is dominated by rounding-error analysis, not by rewriting code.

5. **Pack further, if profiling still demands.** For tight SIMD loops, drop to `int8_t` / `int16_t` and accept that every operation is a contract with the hardware that you have *proven* no input can trigger overflow. The concept that binds the generic code through this hop is `IsPartialAbelianGroup` — explicitly named to record that the operation is partial.

The rule behind the three tables: **you start correct, you finish fast, and the type system enforces that the laws your code relies on are the laws the carrier actually provides — or fails to compile.** The reviewer who points out that `Rational<long>` cannot scale to industrial problem sizes and the reviewer who points out that `double` is not $\mathbb{Q}$ are both right; the library's answer is that one lives on Table 1 and the other on Table 2, and Table 3 spells out the named arrow that moves between them.

4 Sets Are Rules, Not Buckets

4.1 The Comprehension View

Here is the move. A set is not a bucket. A set is a rule:

$$S = \{x \in A \mid P(x)\}.$$

The right-hand side is the definition most mathematicians write down first and then silently forget, reaching instead for a list or a `std::vector`. In `dedekind` we take that right-hand side literally. The set S is the *type* whose inhabitants are elements of the ambient species A satisfying P. Nothing is enumerated, nothing is allocated, nothing is stored. The predicate *is* the set.

That shift—what Lawvere called the comprehension view [3, 20]—is the one idea this section needs. Once you accept it, everything downstream (intersection, complement, emptiness, pruning) falls out as predicate algebra. The subobject classifier Ω becomes a knob you can turn: classical `bool` for exact domains, Kleene K3 for floats, anything else you can register.

```
using namespace dedekind::category;

// ambient_set<T>(P) creates the intensional set {x in T | P(x)}.
```

```
// No values are stored; the predicate IS the set.
constexpr auto even_gt_ten = ambient_set<int>(
    [](const int& x) { return x % 2 == 0 && x > 10; });

// Membership is a function call returning a logic value; no runtime
// container or heap allocation is involved.
static_assert(even_gt_ten(12) == ClassicalLogic::True);
static_assert(even_gt_ten(7)  == ClassicalLogic::False);
```

Listing 4: A set as a compile-time predicate type. The set object is invocable; membership returns a logic value.

4.2 Set Operations as Predicate Composition

If a set is a predicate, a set operation is whatever operation you would apply to the predicates. The dictionary writes itself:

$$S_1 \cap S_2 \triangleq \{x \in A \mid P_1(x) \wedge P_2(x)\}$$
$$S_1 \cup S_2 \triangleq \{x \in A \mid P_1(x) \vee P_2(x)\}$$
$$\overline{S_1} \triangleq \{x \in A \mid \neg P_1(x)\}$$

Every operation on the left is an operation on Ω on the right. Intersection is conjunction; union is disjunction; complement is negation. No values are touched, because there are no values yet. The combinators compose at compile time and produce a new type—a new rule—which the compiler is then free to inspect.

```
using namespace dedekind::category;

// Two intensional sets over int.
constexpr auto evens    = ambient_set<int>(
    [](const int& n) { return n % 2 == 0; });
constexpr auto ten_plus = ambient_set<int>(
    [](const int& n) { return n > 10; });

// Intersection: the predicate of (evens & ten_plus) is the conjunction
// of the two component predicates, built as a type by the compiler.
constexpr auto even_and_ten_plus = evens & ten_plus;

static_assert(even_and_ten_plus(12) == ClassicalLogic::True);
static_assert(even_and_ten_plus(11) == ClassicalLogic::False);
static_assert(even_and_ten_plus(4)  == ClassicalLogic::False);
```

Listing 5: Set intersection as type-level predicate conjunction, discharged by the library's `operator&` on `Set`.

4.3 The Subobject Classifier Ω

Here is what Ω is doing for us. A predicate $P : A \to \Omega$ maps ambient elements to *truth values*, and Ω is the type those truth values live in. Change Ω, and you change what counts as membership.

The default is the one every programmer expects: $\Omega = \{\top, \bot\}$, classical two-valued logic. Plug it in and you get ordinary set theory.

Swap it for Kleene K3—$\Omega = \{-1, 0, +1\}$—and something more interesting happens. The extra value is *Unknown*, and it is where `NaN` lives. In classical logic, $P \vee \neg P = \top$ is a theorem;

with `NaN` in play, that theorem quietly breaks, because neither $x < \texttt{NaN}$ nor $x \geq \texttt{NaN}$ is true. In K3, $P \vee \neg P = \mathit{Unknown}$ is a calm, unremarkable identity. The paradox was never in the floats; it was in our choice of Ω.

That is the payoff: the same comprehension machinery handles exact domains (pick classical Ω) and floating-point domains (pick K3) with no change to the API. The logic is a parameter.

5 Compile-Time Reductions: From Set Pruning to Optimisation and Calculus

5.1 Structural Pruning in Practice

The payoff of the comprehension view is that the LLVM backend can *prune* a set expression to a no-op when its predicate is statically unsatisfiable.

```
using namespace dedekind::order;

constexpr auto n = var<>;

// Two halfspace-structured sets, with pivots (5 and 3) carried as NTTPs.
constexpr auto gt_five  = Set{n % N | (n > bound<5>)};
constexpr auto lt_three = Set{n % N | (n < bound<3>)};

// operator& dispatches through `structured_and`; the type-level
// contradiction rule collapses the meet to the empty-set type ∅<int>.
constexpr ∅<int> empty_meet = gt_five & lt_three;
static_assert(empty_meet == ∅{});
```

Listing 6: Contradictory predicates collapse to the empty-set type ∅. Uses the `Halfspace`-NTTP DSL to pin both bounds in the type signature, so the contradiction is resolved at translation time.

The generated binary contains no membership-test code for `empty_meet`: the type-level contradiction rule resolves the result to ∅ *before* code generation, and the computability tier of ∅ is fully top (extensional/classical/element-in-type). The showcase listings below make this mechanically observable.

Showcase witnesses

Two concrete witnesses from the test suite demonstrate this in the actual `dedekind` API.

Showcase 1 — Diagonal contradiction in $\mathbb{R}^2$. On the diagonal $\{(x, y) \mid x = y\}$, the strip predicate $x > 5 \wedge y < 3$ is contradictory (since $x = y > 5$ forces $y > 5$, violating $y < 3$). The intersection is proven empty by `static_assert`; the exported function compiles to a single `ret i1 false`.

```
constexpr auto R2 = R * R;
constexpr auto xy = var<decltype(R2)>;
using R2Point = typename decltype(R2)::Domain;

// Diagonal: { (x, y) ∈ ℝ² | x = y }
constexpr auto diagonal =
    Set{xy % R2 | [](R2Point p) { return p.first == p.second; }};

// Strip: { (x, y) ∈ ℝ² | x > 5 ∧ y < 3 }
constexpr auto strip = Set{
    xy % R2 | [](R2Point p) { return (p.first > 5.0) && (p.second < 3.0); }};

constexpr auto empty_diagonal_cut = diagonal & strip;
```

```
using R2Logic = typename decltype(empty_diagonal_cut)::logic_species;

static_assert(empty_diagonal_cut(R2Point{6.0, 6.0}) == R2Logic::False);

extern "C" __attribute__((noinline)) bool witness_empty_diagonal_cut() {
  return empty_diagonal_cut(R2Point{6.0, 6.0}) == R2Logic::True;
}
```

Listing 7: Showcase 1 source: diagonal contradiction in $\mathbb{R}^2$.

```
define noundef zeroext i1 @witness_empty_diagonal_cut() local_unnamed_addr #0 {
  ret i1 false
}
```

Listing 8: Showcase 1 LLVM IR (`clang++-22 -O2`): constant false return.

Showcase 2 — Lattice/square singleton in $\mathbb{C}$. The natural-number lattice (Gaussian integers, $0 \leq \mathrm{Re}, \mathrm{Im} \leq 3$) intersected with the square $[0.5, 1.5]^2$ contains exactly $c_3 = 1 + i$. The compiler proves all four membership assertions at translation time and folds the exported function to a single `ret i1 true`.

```
constexpr auto c = var<>;

constexpr auto natural_lattice_in_c =
    Set{c % C | [](const Complex<double>& z) {
      return is_integral_coordinate(z.real()) &&
             is_integral_coordinate(z.imag()) &&
             (z.real() >= 0.0) && (z.real() <= 3.0) &&
             (z.imag() >= 0.0) && (z.imag() <= 3.0);
    }};
constexpr auto square_c1_c2 = Set{c % C | [](const Complex<double>& z) {
  return (z.real() >= 0.5) && (z.real() <= 1.5) &&
         (z.imag() >= 0.5) && (z.imag() <= 1.5);
}};
constexpr auto lattice_square_intersection = natural_lattice_in_c & square_c1_c2;
using CLogic = typename decltype(lattice_square_intersection)::logic_species;

static_assert(lattice_square_intersection(Complex<double>{1.0, 1.0}) == CLogic::True);
static_assert(lattice_square_intersection(Complex<double>{0.0, 1.0}) == CLogic::False);

extern "C" __attribute__((noinline)) bool witness_lattice_square_singleton() { /* ... */
```

Listing 9: Showcase 2 source: lattice/square singleton in $\mathbb{C}$.

```
define noundef zeroext i1 @witness_lattice_square_singleton() local_unnamed_addr #0 {
  ret i1 true
}
```

Listing 10: Showcase 2 LLVM IR (`clang++-22 -O2`): constant true return.

Showcases 1 and 2 are checked in as LLVM IR fixtures and validated in CI (`make ir-fixture-check`). They exemplify the base case: pair-level predicate composition where constant-folding does the work.

From lambdas to structured predicates: the halfspace DSL

Lambda-based predicates let the optimiser fold at specific call sites, but they erase the *structure* of the constraint. A richer pattern emerges when halfspace bounds live in the *type*: $\{n \in \mathbb{N} \mid n > 5\}$ is represented as `Halfspace<int, 5, Up, Strict>`, with the pivot as a non-type template

parameter. Contradiction and cardinality analysis then run in the type system itself, and the reductions are *structural*: no appeal to runtime values is needed.

Showcase 3 — Halfspace contradiction on $\mathbb{N}$. Two opposing halfspaces with non-bridging pivots; the meet reduces to $\varnothing$ by the type-level contradiction rule.

```
constexpr auto n = var<>;

constexpr auto gt_five  = Set{n % N | (n > bound<5>)};
constexpr auto lt_three = Set{n % N | (n < bound<3>)};

// Set-level `&` dispatches through `structured_and` on the halfspace types;
// the contradiction collapses the result's type to ∅<int>.
constexpr ∅<int> empty_meet = gt_five & lt_three;
static_assert(empty_meet == ∅{});

// Computability as a compile-time observable: the parent Sets carry NONE of
// the three tiers; the reduced ∅ carries ALL THREE.
static_assert(!HasDecidableMembership<decltype(gt_five)>);
static_assert(!IsFiniteSet<decltype(gt_five)>);
static_assert(!IsCompileTimeEnumerable<decltype(gt_five)>);
static_assert( HasDecidableMembership<decltype(empty_meet)>);
static_assert( IsFiniteSet<decltype(empty_meet)>);
static_assert( IsCompileTimeEnumerable<decltype(empty_meet)>);
```

Listing 11: Showcase 3 source: halfspace contradiction on $\mathbb{N}$ collapses to $\varnothing$.

```
define noundef zeroext i1 @witness_empty_halfspace_meet() local_unnamed_addr #0 {
  ret i1 false
}
```

Listing 12: Showcase 3 LLVM IR: constant false return.

Showcase 4 — Cardinality-1 reduction on $\mathbb{N}$. The same meet operator, applied to two halfspaces whose bounds pin exactly one integer: the result's type *is* `Singleton<4>`, with the unique inhabitant in the template parameter.

```
constexpr auto n = var<>;

constexpr auto gt_three = Set{n % N | (n > bound<3>)};
constexpr auto lt_five  = Set{n % N | (n < bound<5>)};

// On an integral carrier the meet's cardinality is compile-time-decidable.
// Cardinality 1 elevates the reduction from OrderInterval to Singleton.
constexpr Singleton<4> in_between = gt_three & lt_five;
static_assert(in_between == Singleton<4>{});

// Same computability tightening as Showcase 3, with the element now in the type.
static_assert(!IsCompileTimeEnumerable<decltype(gt_three)>);
static_assert( IsCompileTimeEnumerable<decltype(in_between)>);
```

Listing 13: Showcase 4 source: cardinality-1 meet collapses to $\{4\}$.

```
define noundef zeroext i1 @witness_halfspace_singleton() local_unnamed_addr #0 {
  ret i1 true
}
```

Listing 14: Showcase 4 LLVM IR: constant true return at the unique inhabitant.

The two halfspace showcases make the central claim of the paper mechanically observable: compile-time reduction of an intensional description over a transfinite carrier to a named extensional object (either $\varnothing$ or a `Singleton`) monotonically restores decidability, finiteness, and

type-level knowledge of elements. The *reduction boundary* is the *computability boundary*, and the six `static_asserts` flanking each reduction are its mechanical witness.

Further showcases in the source tree exercise the same machinery on $\mathbb{R}$ ambient carriers (showcase 5), non-point intervals with observable cardinality (showcase 6: $-21 < n \leq 21$ on $\mathbb{Z}$, $|\cdot| = 42$), integer lattice intersected with real-valued bounds (showcase 7), and 2D rectangles via structural cartesian product (showcase 8: $42 \times 11 = 462$). All are IR-verified.

5.2 Linear Programming: The Optimum as a Typed Constant

Stare at a linear program for a moment. The polytope is an intersection of halfspaces—a set, in exactly the comprehension sense of Section 4. The objective is a linear functional. The optimum is the one vertex of the polytope at which the objective is largest.

The previous section's reductions collapsed a set to a simpler set. An optimum is also a collapse: it reduces a set to a single point. The move of this section is to lift that collapse into the type system. If every halfspace and every objective coefficient is named at the type level, the compiler has enough information to enumerate the vertices, run the feasibility checks, compare the objective values, and emit the winner as a literal. The LP solver is the C++ compiler.

Showcase 9 (centrepiece). Consider a textbook LP:

$$\begin{aligned} \text{maximise} \quad & 3x + 2y \\ \text{subject to} \quad & x + y \leq 4 \\ & 2x + y \leq 6 \\ & x, y \geq 0 \end{aligned}$$

The feasible region is a polygon in $\mathbb{Q}^2$ (Figure 2); the optimum is the vertex $(2, 2)$ where the non-axis-aligned constraints $x+y = 4$ and $2x+y = 6$ are simultaneously binding (objective value 10). In `dedekind.optimization:lp`, constraints are NTTPs and the reduction is a variadic template whose output carries the vertex at the type level:

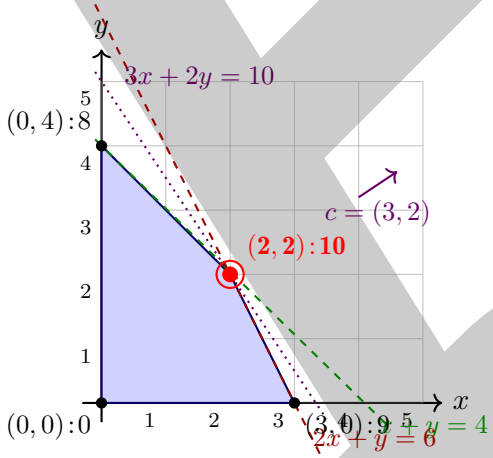

Figure 2: The feasible polygon of Showcase 9. Vertices are labelled with the objective value $3x+2y$ at each. The active-set enumeration considers all $\binom{4}{2} = 6$ pairs of constraints; four pairs produce feasible intersections, and the optimum is the one vertex (red) where both non-axis-aligned constraints $H_1 \colon x+y \leq 4$ and $H_2 \colon 2x+y \leq 6$ bind simultaneously. At translation time the compiler performs this enumeration; the emitted binary contains only the literal coordinates of $(2, 2)$.

```
using Rat = Rational<SignedExtensionalCardinal<>>;  // canonical exact ℚ
using H1 = Halfspace2D<Rat, Rat{1}, Rat{1}, Rat{4}>;     //  x +  y <= 4
using H2 = Halfspace2D<Rat, Rat{2}, Rat{1}, Rat{6}>;     // 2x +  y <= 6
using H3 = Halfspace2D<Rat, Rat{-1}, Rat{0}, Rat{0}>;    //  x      >= 0
using H4 = Halfspace2D<Rat, Rat{0}, Rat{-1}, Rat{0}>;    //       y >= 0

using Optimum = decltype(maximize<Rat, Rat{3}, Rat{2}, H1, H2, H3, H4>());
static_assert(std::same_as<Optimum, Vec2<Rat, Rat{2}, Rat{2}>>);
```

Listing 15: Compile-time LP: the optimum IS a type. From `showcase_09_lp_vertex_typed_constant.cpp`.

The reduction enumerates vertex candidates at compile time (all $\binom{4}{2} = 6$ pairs of binding constraints), solves each 2×2 active-set system via `Invertible2x2`'s closed-form Cramer inverse (exact over $\mathbb{Q}$), filters by feasibility against the non-active constraints, and picks the objective-maximising feasible vertex. Every step is a template specialisation; no runtime iteration survives.

IR witness. The paper's claim is not prose; it is translation-time mechanics. Two `extern "C"` functions in the fixture return the numerator of the optimum's x- and y-coordinates. At `-O2`, `clang`'s optimiser folds the entire LP reduction to literal integers:

```
define noundef i64 @witness_lp_optimum_x() ... {
  ret i64 2
}

define noundef i64 @witness_lp_optimum_y() ... {
  ret i64 2
}
```

Listing 16: Emitted LLVM IR for Showcase 9. The six candidate solves, six feasibility checks, and six objective comparisons collapse to constant returns.

The IR is checked into the repository as a fixture (`showcase_09_lp_vertex_typed_constant.ll`); a build-time gate (`make ir-fixture-check`) fails CI if optimiser drift ever loses the collapse. The binary contains no LP solver; no simplex tableau; no active-set machinery. The vertex *is* a type, and returning its coordinate *is* a literal.

This closes one of the type-directed-collapse axes listed in Section 5.8 in a worked form: the LP solver is the C++ compiler, for every instance whose size and coefficients are compile-time data.

5.3 Forward-Mode AD as a Sibling Realisation of Compile-Time Derivatives

Now swap the ring. Re-run the same LP reduction, but over $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$ instead of $\mathbb{Q}$. Out the other end comes the optimum *and* its derivative with respect to a chosen input—as one typed constant. No separate derivative pass, no finite differences, no re-solve. The chain rule has already run, quietly, as the arithmetic inside the Cramer solve.

The framing is Elliott's [5]: forward-mode AD is the product (f, f') carrying the chain rule on its back, not the "dual number" object *per se*. Dual numbers are simply the realisation of the (value, tangent) product in which $\varepsilon^2 = 0$ makes the chain rule fall out of the ring laws. Seen that way, `dedekind` has been shipping derivatives all along: `dedekind.analysis:ftc::derivative_at` computes the same abstract derivative numerically (central difference), and `dedekind.algebra:polynomial::R` computes it symbolically (exact formal derivative). The new object, `Dual<F>`, is the representation that lives at the type level: its primal and tangent are structural fields, so `Dual<Rational<long>>` slots into any NTTP-parameterised carrier —including the LP's `Halfspace2D<T, a, b, c>`—without special-casing.

Showcase: parametric LP sensitivity. Perturb H_1's bound by ε—replace $x + y \le 4$ with $x + y \le 4 + \varepsilon$. The active set at the optimum is unchanged for infinitesimal perturbation; the Cramer solve now runs over dual numbers:

```
using D = Dual<Rat>;
using H1P = Halfspace2D<D, D{Rat{1L}}, D{Rat{1L}},
                        D{Rat{4L}, Rat{1L}}>;  // bound = 4 + epsilon
using H2D = Halfspace2D<D, D{Rat{2L}}, D{Rat{1L}}, D{Rat{6L}}>;
using H3D = Halfspace2D<D, D{Rat{-1L}}, D{Rat{0L}}, D{Rat{0L}}>;
using H4D = Halfspace2D<D, D{Rat{0L}}, D{Rat{-1L}}, D{Rat{0L}}>;

constexpr auto v = maximize_value<D, D{Rat{3L}}, D{Rat{2L}},
                                  H1P, H2D, H3D, H4D>();
static_assert(v.x.val == Rat{2L}  && v.x.der == Rat{-1L});
static_assert(v.y.val == Rat{2L}  && v.y.der == Rat{2L});
```

Listing 17: Parametric LP over `Dual<Rat>`: optimum + sensitivity as one typed constant. From `lp_test.cpp`.

The primal is the original optimum $(2, 2)$; the tangents $(-1, +2)$ are the partial derivatives of x^* and y^* with respect to the perturbation parameter. A hand derivation confirms: with the active set $\{x + y = 4 + t,\ 2x + y = 6\}$, the solution is $x^*(t) = 2 - t$, $y^*(t) = 2 + 2t$, so the sensitivities are -1 and $+2$. The point of this showcase is not the hand derivation—it is that the library's LP reduction, written once over an unconstrained ring-like carrier, yields exact sensitivity analysis when instantiated over the right ring.

The same `Dual`∘`Rational` layering composes further. Over `Dual<Complex<Rational<long>>>`, the reduction would compute optima and sensitivities over the Gaussian rationals $\mathbb{Q}(i)$; the ring tower is closed under these layerings by construction (every intermediate carrier satisfies the `IsRingLike` concept documented in the companion report). Holomorphic automatic differentiation at compile time is a direct consequence; we leave its application to a follow-up work.

5.4 Linear Algebra: $\mathbb{C}$ and $\mathbb{D}$ as Subrings of $M_2(\mathbb{Q})$

One more sibling, from a slightly different angle. The LP reduction of Section 5.2 used a 2×2 linear algebra layer (`Vec2`, `Invertible2x2`) inside the Cramer solve; the same layer supports something cleaner to exhibit— the compiler verifying an algebraic identity between *scalars* and *matrices*.

Two classical embeddings sit in the library: the complex number $a + bi$ lifts to a rotation-scaling 2×2 matrix, and the dual number $a + b\varepsilon$ lifts to a shear 2×2 matrix:

$$\mathbb{C} \hookrightarrow M_2(\mathbb{Q}) : \quad a + bi \mapsto \begin{pmatrix} a & -b \\ b & a \end{pmatrix}, \qquad \mathbb{D} \hookrightarrow M_2(\mathbb{Q}) : \quad a + b\varepsilon \mapsto \begin{pmatrix} a & b \\ 0 & a \end{pmatrix}.$$

Read left-to-right, these are two sentences in one breath. Read side-by-side, they tell a story: the complex-plane rotation that makes $i^2 = -1$ happen, and the shear that makes $\varepsilon^2 = 0$ happen, are the same *kind* of move—a scalar ring embedded in a matrix ring by a ring homomorphism ϕ satisfying $\phi(xy) = \phi(x)\,\phi(y)$ and $\phi(x + y) = \phi(x) + \phi(y)$. The library claims these identities at the type level, and the compiler discharges them.

Showcase 12 — both homomorphisms, both over $\mathbb{Q}$. The exact carrier is `Rational<SignedExtensionalCa` so all the identities are bit-perfect; no tolerance, no rounding:

```
using Rat = Rational<SignedExtensionalCardinal<>>;  // canonical exact Q

// Complex numbers as 2x2 rotation-scaling matrices.
constexpr Complex<Rat> z{Rat{1}, Rat{2}};
```

```
constexpr Complex<Rat> w{Rat{3}, Rat{-5}};
static_assert(as_matrix2x2(z + w) == as_matrix2x2(z) + as_matrix2x2(w));
static_assert(as_matrix2x2(z * w) == as_matrix2x2(z) * as_matrix2x2(w));

// Dual numbers as 2x2 shear matrices.
constexpr Dual<Rat> d{Rat{2}, Rat{3}};
constexpr Dual<Rat> e{Rat{-1}, Rat{5}};
static_assert(as_matrix2x2(d + e) == as_matrix2x2(d) + as_matrix2x2(e));
static_assert(as_matrix2x2(d * e) == as_matrix2x2(d) * as_matrix2x2(e));

// The defining relation \epsilon^2 = 0 lifts cleanly.
constexpr Dual<Rat> eps{Rat{0}, Rat{1}};
static_assert(as_matrix2x2(eps) * as_matrix2x2(eps) == zero_matrix2x2_v<Rat>);
```

Listing 18: Ring-homomorphism identities for $\mathbb{C}$ and $\mathbb{D}$ into $M_2(\mathbb{Q})$, both discharged as `static_asserts`. From `embeddings_test.cpp`.

The `eps * eps` assertion is the one that lands the pedagogical point. The defining relation of $\mathbb{D}$ is $\varepsilon^2 = 0$; transported through the matrix embedding, it becomes $\left(\begin{smallmatrix}0&1\\0&0\end{smallmatrix}\right)^2 = \left(\begin{smallmatrix}0&0\\0&0\end{smallmatrix}\right)$, an elementary fact of matrix arithmetic. The compiler checks both sides and agrees: the algebraic relation downstairs and the matrix relation upstairs are the same fact. That equivalence, free of narration, is the point.

5.5 Polynomials and the Fundamental Theorem, Verified at Compile Time

Turn the same lens on calculus. A polynomial is a sequence of coefficients, so "differentiate this polynomial" is an operation on sequences—shift down one exponent, multiply each coefficient by its old exponent. The operation is exact, runs entirely in the underlying ring, and the answer is itself a polynomial. If the polynomial lives at the type level (or as a `constexpr` value), so does its derivative. The Fundamental Theorem of Calculus is a sentence in the same language: "differentiation and integration are inverse, modulo a constant of integration," and we would like the compiler to verify it.

Showcase 10 — polynomial derivative, exact. `dedekind.algebra:polynomial` already ships `RigPolynomial<R>::derive()` as a `constexpr` method. Given $p(x) = 3x^3 + 2x^2 + x + 1$ over $\mathbb{Q}$, the library computes $p'(x) = 9x^2 + 4x + 1$ at compile time, and the identity is a type-level assertion:

```
using   = int;                  // the integer coefficient ring
using P = Polynomial <>;

// p(x)  = 3x^3 + 2x^2 + x + 1  (coefficients constant-first)
// p'(x) = 9x^2 + 4x + 1
static_assert(P({1, 1, 2, 3}).derive() == P({1, 4, 9}));
static_assert(P({0, 1}).derive() == P::one());    // d/dx (x) = 1
static_assert(P::one().derive().is_zero());       // d/dx (1) = 0
```

Listing 19: Polynomial derivative: an exact identity, discharged by the compiler. From `polynomials_test.cpp`.

This is the same reduction pattern as the halfspace and LP showcases, applied at the polynomial carrier: the answer is not *computed at runtime and compared to a literal*, it is *the literal in the emitted object file*. Coefficients are exact because `RigPolynomial::derive()` is a ring homomorphism, not a numerical approximation. The `int` carrier in the showcase is the minimal witness; the same identities go through unchanged for `Rational<long>`, and more generally for any `IsRingLike` carrier—so a rational polynomial's derivative is as exact as its coefficients.

Note the pattern: the polynomials are *inlined* at the `static_assert` call site rather than declared as `constexpr` variables. The reason is a transient detail of C++20/23 `constexpr`: `std::vector` may be used inside a constant expression, but its storage must be released before that expression ends, so a namespace-scope `constexpr` polynomial variable with vector-backed coefficients does not compile. Inlining keeps every allocation transient. Replacing the coefficient storage with a fixed-capacity `std::array` would lift this restriction and allow a namespace-scope `constexpr` polynomial variable; we leave that to a follow-up.

Showcase 11 — FTC Part II, numerically witnessed at compile time. The Fundamental Theorem of Calculus, Part II, says that for f continuous on $[a, b]$ with antiderivative F,

$$\int_a^b f(x)\,\mathrm{d}x \;=\; F(b) - F(a).$$

The library's `ftc_part_ii_bridge` is a `constexpr` function that runs a trapezoidal quadrature of f against a supplied F, checks the two sides agree within a tolerance, and returns a `bool`. Pass it a function and its antiderivative as lambdas, and the compiler does the integration:

```
// f(x) = 3x^2;  F(x) = x^3  is an antiderivative,
// so  \int_0^2 3x^2 dx  should equal  F(2) - F(0) = 8.
constexpr auto f = [](double x) { return 3.0 * x * x; };
constexpr auto F = [](double x) { return x * x * x; };

static_assert(ftc_part_ii_bridge<double>(f, F, 0.0, 2.0, 1e-3));
```

Listing 20: FTC Part II, discharged by the compiler on a polynomial f and its closed-form antiderivative F. From `ftc_test.cpp`.

The `static_assert` compiles, which means the compiler ran the full trapezoidal rule (default 4096 slices), subtracted $F(2) - F(0)$, and confirmed the result agreed with 8.0 within tolerance 10^{-3}. As with the LP centrepiece, there is no runtime integration: the numerical verification of a calculus theorem happens during translation, and the binary contains only the literal `true`.

A small engineering aside that reinforces the Compiler-Wall discussion of Section 5.9: lifting this check into `static_assert` required replacing a single `std::abs(double)` call with a hand-rolled `(v < 0 ? -v : v)` inside `detail::magnitude`. Libc++'s `std::abs(double)` is not `constexpr`, which is enough to block the entire `close_enough` chain from compile-time evaluation. Such one-symbol obstructions are the texture of the wall: easy to fix once spotted, but invisible until a `static_assert` exposes them.

What the pair contributes. Showcase 10 is exact-algebraic: the derivative *is* a polynomial, and the identity is a ring-theoretic fact the compiler just agrees with. Showcase 11 is numerical-constructive: the integral is computed, by the compiler, on a concrete interval. Together they pin down the two faces of the derivative already present in the library (exact formal on polynomials, numerical on general functions) as siblings of the dual-number forward AD of Section 5.3. Three realisations of (f, f'), each faithful at the level its carrier allows, and each a `static_assert`-witnessed compile-time reduction in the sense of the present paper.

5.6 Embedded Sensor Fusion: A Kalman Filter as a Compile-Time Primitive

The previous subsections demonstrate the discipline on the mathematician's favourite objects: LP vertices, derivatives, homomorphisms, FTC. This subsection runs the same discipline on the embedded engineer's favourite object. A scalar Kalman filter is the canonical building block of sensor fusion—used in spacecraft navigation from Apollo onward, in every modern IMU-based pose estimator, in essentially every robotics control stack. The runtime cost of a single scalar update step is modest; the cumulative cost, at kHz-scale loops on embedded targets with

battery budgets, is emphatically not. If the filter's coefficients are known at compile time—and in a deployed embedded system they very nearly always are, because they encode sensor characteristics—then every arithmetic operation the filter performs at run time is a candidate for compile-time specialisation.

The filter, in five lines. Given process noise Q and measurement noise R (both known at compile time), state estimate $\hat{x}$ and covariance P (carried at run time as `Rational<SignedExtensionalCardinal<>>`) and a measurement z, one stationary-model update is:

$$
\begin{aligned}
P &\leftarrow P + Q && \text{(predict)} \\
y &= z - \hat{x} && \text{(innovation)} \\
S &= P + R && \text{(innovation covariance)} \\
K &= P / S && \text{(Kalman gain)} \\
\hat{x} &\leftarrow \hat{x} + K \cdot y && \text{(state update)} \\
P &\leftarrow (1 - K) \cdot P && \text{(covariance update)}
\end{aligned}
$$

Five arithmetic operations: one division, three multiplications, two additions and two subtractions, over the underlying ring. No matrix inverse (state is one-dimensional), no square roots (stationary linear model), nothing that blocks exact-rational evaluation.

Showcase 13 — one-step update, compile-time trajectory. The filter is a template parameterised by the two noise constants; `predict` and `update` are `constexpr`:

```
using Rat = Rational<SignedExtensionalCardinal<>>;  // canonical exact ℚ

template <typename R, R Q, R Rnoise>
struct KalmanFilter1D {
  R x_hat;
  R P;

  constexpr void predict()       { P = P + Q; }          // F = 1
  constexpr void update(R z) {
    const R innovation = z - x_hat;
    const R S          = P + Rnoise;
    const R K          = P / S;
    x_hat = x_hat + K * innovation;
    P     = (R{1} - K) * P;
  }
};

// Q = 1/100, R = 1/10, initial estimate 0, initial covariance 1,
// single measurement z = 1.
constexpr auto filtered = [] {
  KalmanFilter1D<Rat, Rat{1, 100}, Rat{1, 10}> f{Rat{0}, Rat{1}};
  f.predict();
  f.update(Rat{1});
  return f;
}();

static_assert(filtered.x_hat == Rat{101, 111});
static_assert(filtered.P     == Rat{101, 1110});
```

Listing 21: A scalar Kalman filter as a compile-time specialised primitive. Filter coefficients are NTTPs; the trajectory is verified by `static_assert` over exact rationals. From `kalman_showcase_test.cpp`.

The arithmetic checks by hand: after `predict`, $P = 1 + \frac{1}{100} = \frac{101}{100}$; the innovation is $y = 1$; $S = \frac{101}{100} + \frac{1}{10} = \frac{111}{100}$; $K = \frac{101}{111}$; $\hat{x}' = \frac{101}{111}$; $P' = \frac{10}{111} \cdot \frac{101}{100} = \frac{101}{1110}$. The compiler verifies this exact sequence at translation time, and a companion two-step test asserts covariance contraction, $P_{\text{after two updates}} < P_{\text{after one update}}$, as a compile-time fact.

Why this matters for embedded deployment. The showcase's value is not the one-step numerical result but the *shape* of the code it represents. In a production embedded system, Q and R are sensor-characterisation constants written into firmware; $\hat{x}$ and P are runtime state; z is a runtime sample. The `constexpr` structure specialises once at build time to the specific filter a given sensor requires, and the run-time update is an allocation-free, dispatch-free, branch-free sequence of primitive arithmetic that any mainstream embedded toolchain will schedule tightly. On carriers where exact rationals are not wanted (`float` on an MCU), the same template instantiates with the carrier-appropriate arithmetic; on carriers where they are (safety-critical replay, bit-exact cross-platform determinism), `Rational<long>` or `Rational<ExtensionalCardinal<>>` gives the reproducibility guarantee built in.

What this showcase does *not* demonstrate: a multi-dimensional state (would require the linear-algebra layer's matrix inversion beyond 2×2); a steady-state solve via the discrete algebraic Riccati equation (the square root blocks exact rationals); or benchmarks against an embedded ML runtime (a follow-up paper's territory). It demonstrates the primitive and the pattern. The rest, where Axiomatic Systems Programming succeeds or fails for embedded ML, is an engineering program we name and invite, not one this paper discharges on its own.

5.7 Algebraic Properties Enable Stronger Pruning

Beyond interval arithmetic, *algebraic* properties of the predicates provide additional pruning opportunities. The `:species` registry records which operations are associative, commutative, idempotent, and so forth. These traits inform the compiler about structural relationships between sets:

- $S \cap \varnothing = \varnothing$ (intersection with the empty set is empty).
- $S \cap \overline{S} = \varnothing$ (intersection with the complement is empty).
- If $S_1 \subseteq S_2$ then $S_1 \cap S_2 = S_1$ (subset absorption).

Each of these equalities can be discharged as a `static_assert` once the relevant algebraic witnesses are in the trait registry, reducing the intersection to the simpler set at compile time.

5.8 From Cardinality Pruning to Type-Directed Collapse

Step back from the showcases for a moment. What is the pattern? In every one of them we started with an intensional set—a predicate—and we ended with a named extensional object whose *type* carries the answer: $\varnothing$, a singleton, an order interval of known cardinality, an LP vertex. The collapse is not incidental; it is a rewrite rule, running inside the type system, at translation time. To make the claim precise, we first formalise "how much the compiler knows about a set" as a lattice, then state the theorem as monotone motion up that lattice.

Definition 1 (Computability tier)**.** *For a compile-time set S over an ambient carrier T, the* computability tier *is the triple*

$$\mathcal{T}(S) \;=\; \big(\,\mathsf{R}(S),\ \mathsf{L}(S),\ \mathsf{K}(S)\,\big) \;\in\; \{\mathrm{Int}, \mathrm{Ext}\} \times \{\mathrm{K3}, \mathrm{Cl}\} \times \{\mathrm{Opq}, \mathrm{Elt}\}$$

whose components are the representation *tier (*R*: intensional vs. extensional, i.e. predicate vs. elements-in-type), the* logic *tier (*L*: Kleene three-valued vs. classical two-valued membership),*

and the knowledge *tier (*K*: elements opaque vs. carried in the type). Each axis carries the order* Int $\preceq$ Ext*,* K3 $\preceq$ Cl*,* Opq $\preceq$ Elt*; the product order is componentwise. The resulting Boolean lattice* $(\mathcal{T}, \preceq)$ *has bottom* $\bot = (\text{Int}, \text{K3}, \text{Opq})$ *("a raw predicate over a floating-point carrier with no inhabitants known") and top* $\top = (\text{Ext}, \text{Cl}, \text{Elt})$ *("a named finite set whose members are template parameters and whose membership is decidable").*

Every showcase of Section 5 is a motion inside this lattice. The empty-halfspace reductions of Showcase 3 go $(\text{Int}, \text{K3}, \text{Opq}) \mapsto \top$ in one step (the result is $\varnothing$); the cardinality-1 reduction of Showcase 4 and the LP reduction of Showcase 9 do the same. The reductions never slide downward. With that in hand:

Proposition 1 (Type-Directed Collapse, halfspace case; empirically verified)**.** *Let* T *be an ordered carrier, and let* P, Q *be halfspace predicates over* T *with pivots carried at the type level as non-type template parameters. The meet*

$$S \;=\; \{x \in T \mid P(x)\} \cap \{x \in T \mid Q(x)\}$$

is resolved by the library at compile time to one of $\varnothing$*, a* `Singleton<v>`*, or an* `OrderInterval` *whose cardinality (when* T *is integral) is a compile-time constant. Writing* $S_1 = \{x \mid P(x)\}$ *and* $S_2 = \{x \mid Q(x)\}$*, the reduction is monotone up the computability-tier lattice:*

$$\mathcal{T}(S) \;\succeq\; \mathcal{T}(S_1) \vee \mathcal{T}(S_2),$$

and strictly so on at least one axis in every case we exhibit.

Status of claim. *This proposition is* not *proved inductively in the paper. It is discharged by exhibition: each case is checked into the library as a* `static_assert`*/*`STATIC_CHECK` *witness and, where relevant, as an LLVM IR fixture that fails CI on optimiser drift (see Section 5). Read the proposition as an empirically verified structural claim about the library, not as a universally quantified theorem; a general inductive proof over pivot configurations is left open.*

Corollary 1 (Iterated collapse)**.** *Any finite composition of type-directed-collapse reductions is itself monotone up* $\mathcal{T}$*. In particular, no sequence of library operations can ever return a set that is* less *decidable than its inputs.*

So: not "a faster way to do set algebra", but a rewrite rule that happens to live in the C++ type system, and whose action is monotone-ascending in a lattice of decidability. The halfspace case is the smallest non-trivial instance we know how to prove mechanically. Four further axes appear to obey the same pattern; the project tracker carries one existential proof per axis—one worked instance, one IR fixture, one compile-time witness—rather than a general framework. We would rather ship one reduction that fully collapses than four sketches that do not.

- **Carrier tightening.** Intersecting a subset of $\mathbb{N}$ with a subset of $\mathbb{R}$ should land in $\mathbb{N}$: the carrier lattice contributes its own structural reduction via the existing canonical embeddings (`embed__`, `embed_`$\mathbb{Q}$`_`$\mathbb{R}$, …).
- **Smooth optimization.** The stationary set $\{x \mid \nabla f(x) = 0\}$ for a quadratic f collapses to a `Singleton<x*>` with the minimizer in the template parameter. Same reduction pattern, applied at the extremum operator.
- **Linear programming** (demonstrated in Section 5.2 and Section 5.3). `maximize(c, polytope)` reduces, for compile-time-sized LPs, to a typed vertex via type-level active-set enumeration, and the same reduction over a dual-number carrier yields sensitivity analysis with no extra code path: the solver is the compiler.

- **Lattice-law rewriting.** Distributivity and absorption applied at the type level rewrite $(A \cup B) \cap \neg A$ to $B \cap \neg A$ *before* local reduction rules fire, exposing collapses that a purely local analysis would miss.

Each axis sits on the foundation shared with Section 5: NTTP predicates, structured `operator&` dispatch, and the computability tiers visible in the showcases. Taken together they sketch a program of which the current halfspace reductions are the first installment.

The polynomial-calculus showcase of Section 5.5 belongs to the same family but at the *function* layer rather than the set layer: a function (polynomial, or a C^∞ lambda) is an intensional object reduced, under a ring homomorphism (`derive`) or a numerical integrator, to another intensional object whose identity with a target is discharged by the compiler. The computability-tier framing lifts to this setting: the derivative of `P({1, 1, 2, 3})` *is* `P({1, 4, 9})` as a `static_assert`-visible value, and `ftc_part_ii_bridge` lifts a calculus theorem from "checkable at runtime" to "true at translation time".

5.9 The Compiler Wall: Where This Stops

Treating `clang++` as a poor man's theorem prover is useful precisely because the compiler is bounded, predictable, and cheap. It is also, for the same reason, incapable of doing everything a real proof assistant would. The purpose of this subsection is to be honest about the wall and to name the `clang` knobs that push it back by a constant factor but cannot remove it.

Template instantiation depth. Every reduction in this paper is written as a variadic template whose depth grows with the number of type arguments. `clang`'s default instantiation-depth limit is 1024; on deep recursions (e.g. an LP with many halfspaces, or the cartesian-product carriers of Showcase 8) we see it earlier than expected because concepts and NTTP lambdas each add their own frames. The escape hatch is `-ftemplate-depth=N`; the `dedekind` build raises it to 2048 on the affected translation units. This buys depth linearly in N at the cost of compile time and resident memory, and it does not help when the recursion is superlinear in the instance size.

`constexpr` step and call-depth limits. `-fconstexpr-steps` (default $\sim$ 1,048,576 on recent `clang`) caps the number of evaluation steps a single `constexpr` computation may take; `-fconstexpr-depth` caps call recursion independently. Both matter here because the LP reduction performs its Cramer solves and feasibility checks inside `consteval` functions. A 2×2 LP with four constraints is comfortably under the default; a 2×2 LP with thirty constraints ($\binom{30}{2} = 435$ active sets) begins to approach the limit with any nontrivial constraint metadata. Raising the cap is a compile-time cost; the more principled answer—and the one we prefer—is to stop at the boundary and hand off to a runtime solver, which is exactly the case our centrepiece is *not*.

Combinatorial blowup is the pessimistic bound. Compile-time vertex enumeration over n halfspaces is, in the worst case, $\binom{n}{d}$ active-set solves for a d-dimensional LP; in the 2D case this paper demonstrates, $\binom{n}{2}$. For $n = 4$ that is six solves; for $n = 20$, 190; for $n = 1000$, nearly half a million. Taken at face value the figure looks alarming. Two facts about real constraint lists soften it substantially.

First, by the fundamental theorem of linear programming, the optimum of a d-dimensional LP is attained at a vertex with at most d binding constraints. In practice the active set is often significantly smaller than d—many vertices of a handwritten polytope bind only one or two of the stated constraints, and most of the syntactic constraint list is either slack at the optimum or logically redundant. The $\binom{n}{d}$ figure is the enumeration cost, not a measure of what any vertex actually witnesses.

Second, many redundancies are *statically* removable when constraints live at the type level. Two halfspaces with the same normal reduce at compile time to the tighter bound: `x > 5 && x > 7` collapses to `x > 7` (stricter pivot wins), `x > 5 && x < 2` collapses to $\varnothing$ (contradiction), and `x > 3 && x < 5` over an integral carrier collapses to `Singleton<4>`. The library's `structured_and` dispatch already performs exactly these reductions for 1D halfspaces (Showcases 3 and 4 are the empty-meet and cardinality-1 witnesses); the same dispatch pattern extends naturally to 2D halfspaces whose normals are linearly dependent. A structurally pruned constraint list is typically much smaller than its syntactic form, and the combinatorial term tracks the pruned count, not the raw one.

For the embedded regime this paper targets—MPC steps with a handful of physical constraints, sensor-fusion guards with a few axis-aligned bounds, parametric configuration LPs—the raw constraint count rarely exceeds a few dozen, and compile-time pruning typically drops it further. The method remains tractable for the instance regime it was built for. The intended partition is unchanged: compile-time reduction for problems whose structure is fixed in the type, runtime simplex or interior-point solvers (and the attendant numerical compromises) for problems whose size or coefficients are only known at execution. Industrial-scale LP is emphatically the latter; we do not claim it is the former. A genuine pathological case would require a constraint list that the author was *unable* to prune structurally—an unlikely shape for the use cases this library is written against.

Numeric overflow in the carrier. The paper's code listings ship with `Rational<SignedExtensionalCardinal` an arbitrary-precision signed rational whose exact range easily accommodates every arithmetic operation the showcases exercise. Cramer's rule over $\mathbb{Q}$ is exact, and on this carrier no determinant — however fat — ever overflows silently; escalation to $\aleph_0$ is tracked in Section 3.4's Rosetta table.

A programmer who trades the exact carrier for a native one (`Rational<long>`, or `double`, per Table 3) inherits the corresponding failure mode: signed-integer overflow is undefined behaviour on the former, rounding is non-associative on the latter. That trade is deliberate and the Rosetta table spells out the named arrow for making it. The crucial property the showcases demonstrate — compile-time reduction to a typed constant — does not depend on carrier choice; it survives every row of the table. Every combinator in Section 5 is written against the `IsRingLike` concept, not against any particular carrier, so the swap between exact and native is a one-line change at the `using` declaration.

Error messages and the readability tax. A failed `static_assert` in a deep reduction produces, by default, a multi-page expansion trail that is difficult for a new user to parse. The mitigation we adopt is a combination of `requires`-clauses at call sites (so the first failing predicate is the reported one), named concepts with short `diagnostic` strings, and targeted `static_assert` messages inside internal reductions. This is an ergonomics concern rather than a correctness one, but it is the concern a reader most often encounters first and deserves explicit acknowledgement.

In short: the compiler wall has a predictable shape—depth, steps, combinatorics, numeric range, diagnostics—and each is either pushable by a flag, avoidable by staying within the instance regime for which this technique is intended, or a known limitation we do not claim to have solved. The paper's contribution lies entirely to the left of that wall.

6 Related Work

Functional and dependently-typed approaches. Agda [7] and Coq provide dependent type systems in which set-membership proofs are first-class terms. The expressiveness is greater

than C++23, but the runtime model differs fundamentally: proof terms are erased at runtime, whereas `dedekind`'s structural pruning targets machine-code generation in a systems-programming context. Haskell [6] supports type-class-based structural reasoning, and libraries such as `algebra` encode ring and field hierarchies. C++ templates differ in that they operate on concrete machine types (with hardware-level algebraic imperfections) rather than erased polymorphic dictionaries.

Exact arithmetic. The GNU Multiple Precision library (GMP) and `mpfr` support arbitrary-precision arithmetic at runtime. `iRRAM` [21] and similar libraries implement computable real arithmetic. `dedekind` is complementary: exact reals (Dedekind cuts) are modelled *intensionally as types*, and realization to a specific arithmetic backend is an explicit programmer choice.

Set-builder type systems. Refinement types [22] attach predicates to types, enabling a form of set-builder representation in ML-family languages. Liquid Haskell [23] extends Haskell with SMT-backed refinement predicates. `dedekind` achieves a subset of this functionality in standard C++23 without a custom type checker, using NTTP-embedded lambdas as the predicate carrier.

Linear programming. The standard LP toolchain—simplex [24] and interior-point methods [25] as implemented in Gurobi, CPLEX, GLPK, and related solvers—treats the problem instance as runtime data: the objective vector, constraint matrix, and right-hand side are passed to a solver that iterates to a vertex. `dedekind` sits orthogonally: problem instances whose size and coefficients are *compile-time data* reduce via type-level vertex enumeration to the optimum as a typed constant (Section 5.2), with the entire solver collapsing out of the emitted binary. This is not a replacement for runtime LP—for large, data-dependent instances the classical pipeline remains indispensable—but a complement for instances that appear during code generation (problem-shape metadata, compile-time configuration polytopes, template-parametric optimizer settings). Exact-rational arithmetic over $\mathbb{Q}$ connects the approach to the exact-LP literature [26]; the novel claim is the realisation at translation time with zero runtime overhead, witnessed in the emitted LLVM IR fixtures.

Automatic differentiation. Runtime AD libraries such as Ceres [10], Adept [27], Sacado, dco/c++, and CasADi [28] provide forward- and reverse-mode AD via expression templates or operator overloading on a specialised arithmetic type. The theoretical view taken by Elliott [5]—that forward-mode AD is the (value, tangent) product with chain-rule composition, not the dual-number object *per se*—clarifies why the same reduction that solves the LP also computes its sensitivities when instantiated over the right ring (Section 5.3). `dedekind`'s contribution is not a new AD algorithm but the observation that a single compile-time reduction, written once over an `IsRingLike` carrier, serves as LP solver over $\mathbb{Q}$ and as forward-mode AD engine over $\mathbb{Q}[\varepsilon]/(\varepsilon^2)$ with no extra code path.

Embedded machine-learning runtimes and edge computing. The embedded / edge-computing [12] landscape has a mature runtime ecosystem that `dedekind` deliberately sits *beside* rather than competes with. TensorFlow Lite Micro [13] compiles models to a small-footprint interpreter targeted at microcontroller-class devices; ONNX Runtime and microTVM generate target-specific code from a model graph; the Eigen linear-algebra library is the de facto substrate for embedded numerics. These runtimes take the model as input and the target as fixed; they do not treat the hosting application's algebraic invariants as proof obligations for the C++ compiler to discharge.

A deliberate choice on the linear-algebra backend is worth flagging here, since it connects the library's position to the abstract-naturals-versus-IEEE-numerics tension of Section 3.2.

Eigen [29] is mature, widely deployed, and excellent at what it targets—but it is also *carrier-opinionated*: its matrix types default to `float` or `double` and its optimisations assume a field-like IEEE carrier. A natural path for `dedekind`'s runtime linear-algebra layer is instead toward GraphBLAS [30, 31], whose explicit *(carrier, +, ×)* semiring parameterisation makes the ring-over-which-matrices-live a first-class configurable property. That matches the discipline this paper argues for: matrices over `bool` (with OR/AND) are perfectly well-defined objects when inversion is not required, matrices over `char` are useful for quantised embedded inference, and both sit naturally inside the `IsRingLike` / `IsSemiringLike` concept hierarchy. The cost of this generality is library complexity rather than carrier simplicity. The trade we advocate is correctness-and-performance (tight control over the ring structure) at the price of implementation effort, against Eigen's opposite trade—an ergonomic default carrier whose optimisations are largely locked to floating-point semantics.

`dedekind` addresses a different surface: the *inline* numerical primitives (Kalman filters, pose-estimation math, MPC steps, parametric optimisation) a robotics, automotive, or instrumentation engineer writes directly in their control loop. For these, a fixed-topology LP, a compile-specialised Kalman update (Section 5.6), or an exact-rational sensor-fusion step expressed against `IsRingLike` concepts exhibits three properties the model-runtime path does not aim at: a binary with no interpreter, a dependency graph no heavier than the system's `clang++`, and bit-for-bit deterministic arithmetic across platforms when the carrier is $\mathbb{Q}$. We see this as a complement to the runtime ecosystem, not a replacement: model-driven inference continues to belong to TFLite Micro and its siblings, while the numerical glue around such inference—the Kalman front-end, the control-loop LP, the gradient for online adaptation—is where Axiomatic Systems Programming is strongest.

Data-pipeline DSLs. Apache Spark, Flink, and the Python `pandas`/`polars` ecosystem provide declarative data pipelines with runtime domain checks. None of these ground their APIs in formal set theory, so intersections and filters are runtime pipeline operations rather than compile-time type invariants. `dedekind` is positioned one layer down: it is a systems-programming library that could in principle back such pipelines, but the contribution we demonstrate here is the compile-time reduction at the C++ layer itself.

C++ template metaprogramming lineage. Compile-time computation in C++ has a long tradition that `dedekind` builds on rather than supplants. Boost.Hana [8] and `mp11` [32] provide compile-time heterogeneous containers and type-level algorithm support; Hana in particular already demonstrates concept-based structural dispatch at compile time and would be a natural lower-layer substrate for several of our combinators. CTRE [9] shows that serious computation (regular-expression matching, compiled to a type-level automaton) can live entirely inside the type system and emit optimised machine code; the shape of that reduction—intensional description to a named extensional object at translation time—is close kin to our halfspace and LP reductions, differing in subject matter but not in mechanism. Expression-template AD libraries including ENOKI [11] and the runtime AD systems Ceres [10], Adept [27], and CasADi [28] have carried pieces of the dual-number story into production workloads.

`dedekind`'s position relative to this lineage is specific and narrow: it targets *algebraic-semantic* content—ring homomorphisms, structural cardinality, ordering laws, the (f, f') product as an Elliott [5] sibling—rather than the syntactic structure manipulation that is the traditional focus of TMP libraries. The contribution is not another metaprogramming trick but the claim, supported by worked reductions, that the *discipline* of treating mainstream C++23 as a structural-gate theorem prover (Axiomatic Systems Programming) is now coherent enough to practise end-to-end.

7 Conclusion

The thesis of this paper, once more: *named algebraic structure at the type level, discharged by the compiler, narrows the gap between abstract mathematics and machine code without a separate proof assistant.* `dedekind` is the existence proof. Sets are rules rather than buckets; intensional descriptions defer materialisation to explicit boundaries; the LLVM backend carries discharged proofs through to object-code optimisation. The centrepiece existential proof (Section 5.2) shows a textbook linear-programming instance reducing at compile time to its optimal vertex as a typed constant—the six candidate solves, six feasibility checks, and six objective comparisons vanish in the emitted IR. The same reduction over the dual-number ring recovers exact forward-mode sensitivities (Section 5.3); over `RigPolynomial` it produces exact symbolic derivatives and discharges the Fundamental Theorem of Calculus at translation time (Section 5.5); at the linear-algebra layer it certifies the ring homomorphisms $\mathbb{C}, \mathbb{D} \hookrightarrow M_2(\mathbb{Q})$ as compile-time facts (Section 5.4). In every case the binary contains only literals.

Axiomatic Systems Programming as a practised discipline. The viewpoint we name in Section 1—treating `clang++` as a poor man's theorem prover, and the `IsRingLike` concept as a proof obligation the carrier must discharge—is the paper's intended export. Taken seriously, it turns each algebraic invariant into a structural gate the compiler polices; taken habitually, it shifts the library author's default from "runtime check" to "type-level obligation". Three axes of type-directed collapse remain open (Section 5.8): carrier tightening, smooth optimization, and lattice-law rewriting; each sits on the same NTTP-plus-concepts foundation as the halfspace and LP reductions demonstrated here. We leave them as worked targets for successor libraries that adopt the same discipline.

A forward claim, in its weakest defensible form. As numerical workloads saturate available hardware, the marginal returns of brute-force optimisation are diminishing, and the lever with a different cost curve is specialisation at translation time rather than run time. That lever also has a sustainability dimension: cycles not spent at run time are watt-hours not drawn from the grid, and the machine-learning research community has begun to measure its compute footprint in exactly those terms [33]. The operating point at which the lever compounds most visibly is not the data centre but the edge: CPU-first and embedded machine-learning deployments [12, 13], where the problem topology is fixed, the runtime footprint is under hard constraint, and the toolchain of least resistance is the system's `clang++`. We do not prove here that axiomatic discipline applied at library scale yields a meaningful fraction of cycles back at either end of the spectrum; we claim only that the ingredients—compile-time verification, structural optimisation, ring-aware code generation, exact arithmetic carriers—are now assembled in mainstream C++23, and that the discipline is worth practising precisely because its payoffs accrue at scales larger than any single library. If even a small fraction of the arithmetic done in contemporary edge deployments—model-predictive control in vehicles, Kalman-style sensor fusion in robotics, gradient evaluations in micro-inference loops—ran against libraries whose algebraic invariants had been discharged upstream, the compounded effect would be difficult to dismiss. The shape of that opportunity is what Axiomatic Systems Programming claims. Whether the shape materialises into the numbers it suggests is an engineering question, and one the next decade of library authors will answer.

References

[1] *<Programming>: The Art, Science, and Engineering of Programming.* `https://programming-journal.org/`. Open-access journal for programming-related research; ISSN 2473-7321.

[2] The dedekind Authors. ***dedekind**: Axiomatic Systems Programming in C++23*. https://github.com/vincentk/dedekind. Open-source implementation, Apache-2.0 licensed. 2026.

[3] F. William Lawvere. "An elementary theory of the category of sets". In: *Proceedings of the National Academy of Sciences* 52.6 (1964), pp. 1506–1511.

[4] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Provides the formal specification for floating-point formats and operations. IEEE. 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[5] Conal Elliott. "The simple essence of automatic differentiation". In: *Proceedings of the ACM on Programming Languages* 2.ICFP (2018), pp. 1–29.

[6] The GHC Development Team. *The Glasgow Haskell Compiler User's Guide, Edition 2024*. GHC 2024 Language Edition. 2024. URL: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html.

[7] Ulf Norell. "Towards a practical programming language based on dependent type theory". PhD thesis. Chalmers University of Technology, 2007. URL: http://www.cse.chalmers.se.

[8] Louis-Dionne Chagnon. "Boost.Hana: Your Metaprogramming Nightmare". In: *CppCon 2015*. https://github.com/boostorg/hana. 2015.

[9] Hana Dusíková. "Compile-Time Regular Expressions". In: *CppCon*. https://github.com/hanickadot/compile-time-regular-expressions. 2018.

[10] Sameer Agarwal, Keir Mierle, and The Ceres Solver Team. *Ceres Solver*. https://github.com/ceres-solver/ceres-solver. 2022.

[11] Wenzel Jakob. "Enoki: Structured Vectorization and Differentiation on Modern Processor Architectures". In: https://github.com/mitsuba-renderer/enoki. 2019.

[12] Weisong Shi et al. "Edge Computing: Vision and Challenges". In: *IEEE Internet of Things Journal* 3.5 (2016), pp. 637–646.

[13] Robert David et al. "TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2021.

[14] Jeff Bezanson et al. "Julia: A Fresh Approach to Numerical Computing". In: *SIAM Review* 59.1 (2017), pp. 65–98. DOI: 10.1137/141000671.

[15] Wenzel Jakob. *nanobind: tiny and efficient C++/Python bindings*. https://github.com/wjakob/nanobind. 2022.

[16] William Shakespeare. *The Most Excellent and Lamentable Tragedy of Romeo and Juliet*. Act 2, Scene 2. John Danter, 1597.

[17] Roger Penrose. *The Road to Reality: A Complete Guide to the Laws of the Universe*. Chapter 16 and surrounding material on Gödel's incompleteness theorem. London: Jonathan Cape, 2004.

[18] Alan M. Turing. "Systems of Logic Based on Ordinals". In: *Proceedings of the London Mathematical Society*. Second Series 45 (1939), pp. 161–228.

[19] Zoran Majkić. "Intensional First-Order Logic for P2P Database Systems". In: *Journal on Data Semantics* 12 (2009), pp. 47–66. DOI: 10.1007/978-3-642-00684-6_4. URL: https://dblp.org/rec/journals/jods/Majkic09.

[20] F. William Lawvere and Robert Rosebrugh. *Sets for Mathematics*. Discusses the construction of the continuum and Dedekind cuts within a categorical framework. Cambridge University Press, 2003.

[21] Norbert Th. Müller. “The iRRAM: Exact Arithmetic in C++”. In: *Computability and Complexity in Analysis (CCA 2000)*. Vol. 2064. Lecture Notes in Computer Science. Springer, 2001, pp. 222–252.

[22] Tim Freeman and Frank Pfenning. “Refinement Types for ML”. In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 1991, pp. 268–277.

[23] Niki Vazou et al. “Refinement Types for Haskell”. In: *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 2014, pp. 269–282.

[24] George B. Dantzig. *Linear Programming and Extensions*. Princeton, NJ: Princeton University Press, 1963.

[25] Narendra Karmarkar. “A New Polynomial-Time Algorithm for Linear Programming”. In: *Combinatorica* 4.4 (1984), pp. 373–395.

[26] Ambros Gleixner, Daniel E. Steffy, and Kati Wolter. “Iterative Refinement for Linear Programming”. In: *INFORMS Journal on Computing* 28.3 (2016), pp. 449–464.

[27] Robin J. Hogan. “Fast Reverse-Mode Automatic Differentiation Using Expression Templates in C++”. In: *ACM Transactions on Mathematical Software* 40.4 (2014), 26:1–26:16.

[28] Joel A. E. Andersson et al. “CasADi: A Software Framework for Nonlinear Optimization and Optimal Control”. In: *Mathematical Programming Computation* 11.1 (2019), pp. 1–36.

[29] Gaël Guennebaud, Benoît Jacob, et al. *Eigen v3: A C++ Template Library for Linear Algebra*. `https://eigen.tuxfamily.org`. 2010.

[30] Jeremy Kepner and John Gilbert. *Graph Algorithms in the Language of Linear Algebra*. Software, Environments, and Tools. SIAM, 2011.

[31] Timothy A. Davis. “Algorithm 1000: SuiteSparse:GraphBLAS: Graph Algorithms in the Language of Sparse Linear Algebra”. In: *ACM Transactions on Mathematical Software* 45.4 (2019), 44:1–44:25.

[32] Peter Dimov. *`mp11`: A C++11 Metaprogramming Library*. `https://github.com/boostorg/mp11`. 2017.

[33] Roy Schwartz et al. “Green AI”. In: *Communications of the ACM* 63.12 (2020), pp. 54–63.